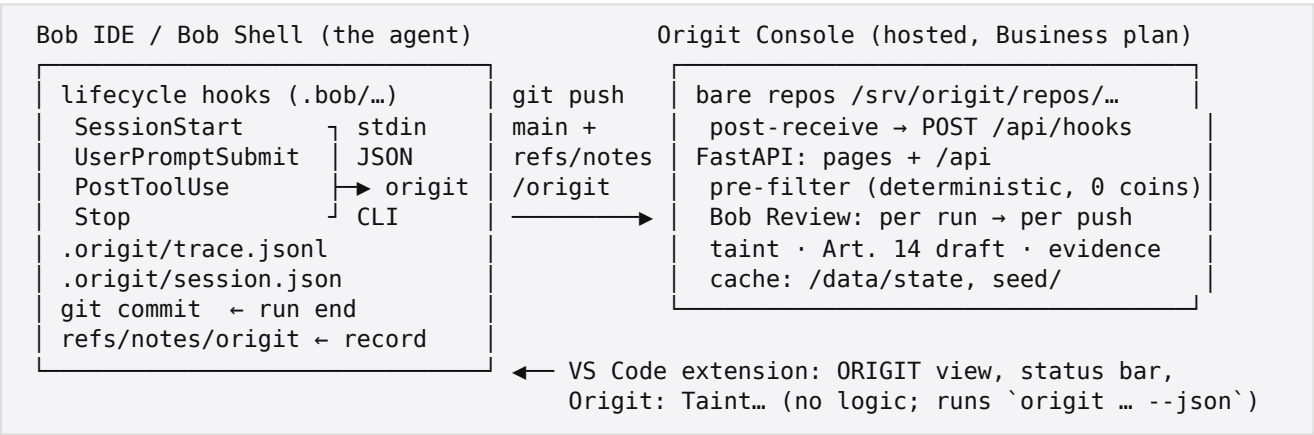


Origit — technical documentation

Origit is an agent provenance layer for git: a hashed record per commit of what an AI coding agent read, wrote, added and ran, stored in git notes and queryable. This document describes the three deliverables as built for the IBM Bob 2.0 Hackathon (September 2026): the open-source core (origit), the hosted console (origit-console, <https://origit.uk>) and the Bob IDE extension (extensions/origit-vscode). Everything here is true of the code at the time of writing; where the code is the spec, the module docstring is cited.

Contents: [1 Architecture](#) · [2 Repositories](#) · [3 Origit core](#) · [4 Bob IDE integration](#) · [5 Demo repository](#) · [6 Origit Console](#) · [7 Extension](#) · [8 Security model and claims](#) · [9 Operations](#)

1. Architecture



Design rule: **deterministic core, AI at the edges**. Nothing under origit/origit/ calls a model or the network. IBM Bob is (1) the agent whose activity is recorded, (2) the reviewer that writes cited OWASP evidence, (3) the drafter of the CRA Article 14 early warning, and (4) one of the builders. Bob never edits a record or a taint result.

Data model in one line: **one Bob session → many runs; one run = one commit = one record** (git note under refs/notes/origit, keyed by commit sha).

2. Repositories

Repository	Purpose	Key paths
origit (public, MIT) https://github.com/cvikl/origit	Core CLI, templates installed by origit init, VS Code extension, docs, Bob session evidence	origit/origit/*.py, origit/tests/, extensions/origit-vscode/, docs/, bob_sessions/, demo/ (submodule + evidence pack), tools/bob/run-task.sh
origit-demo-payments-api	The fictional fintech's repository Bob works in; vendors fast-pay-utils 2.0.0 (clean) and 2.1.0 (compromised, synthetic)	src/, packages/fast-pay-utils/, .bob/, .github/, .origit/config.json

Repository	Purpose	Key paths
origit-console	Hosted web console (FastAPI), deploy files, prompts, seeded Bob output	app/, prompts/, seed/, deploy/, vendor/origit (git submodule of the core)

The console vendors the core as a submodule so CLI and console always share the record format, the pre-filter rules and the taint engine.

3. Origit core

Python 3.11+, single dependency `click`. Install: `cd origit && pip install -e .`. Tests: `cd origit && env -u PYTHONPATH PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 .venv/bin/python -m pytest -q (76 tests)`.

3.1 The record (`record.py`)

Schema `origit/record/v1`. One JSON object per commit:

Field	Content
<code>session</code>	<code>{id, number, run, started_at, ended_at, prompt}</code> — Bob session id (32 hex), display number (#42), run counter inside the session, first 200 chars of the prompt
<code>actor</code>	<code>{kind: bob-ide human, mode, model, config_sha256}</code> — mode slug (origit-build), sha256 over every file under <code>.bob/</code> in effect
<code>read[]</code>	<code>{kind: file pkg url mcp, ref, sha256}</code> — every input the agent consumed; sha256 is the content hash at fold time
<code>wrote[]</code>	paths written (union of traced write tools and files staged in the commit)
<code>added_deps[]</code>	<code>{name, version, registry, lockfile_sha256}</code> — from npm install commands and from the <code>package.json</code> diff (local file: deps resolved to the installed version)
<code>commands[]</code>	shell commands executed, in order
<code>author, approver, approved_at</code>	git user, git config <code>origit.approver</code> , fold time
<code>tests</code>	<code>{run, passed, failed}</code> parsed from the agent's own test run (jest/pytest totals) or from the test suite run by the Stop hook
<code>record_sha256</code>	SHA-256 of the canonical bytes of everything above

Canonical form: JSON, UTF-8, keys sorted recursively, separators `,/:`, `ensure_ascii=False`; `read`, `wrote`, `added_deps` are de-duplicated and sorted (order-independent hash); `commands` keeps its order. `verify(record)` recomputes the hash; the console recomputes it on every view ("hash verified").

3.2 Tracing (`trace.py`)

Bob IDE and Bob Shell emit one JSON payload per lifecycle event on the hook's stdin. Live field names (Bob IDE 2.2, verified 26 Sep 2026): `hook_event_name`, `session_id`, `cwd`, `tool_name`, `tool_input`, `tool_response`, `tool_use_id`, `prompt` (UserPromptSubmit), `last_assistant_message`

(Stop). `unwrap()` also accepts the documented names (`event`, `tool`, `input`, `output`) and the fallback tracer's `{"ts", "raw": {...}}`.

Events are appended to `.origit/trace.jsonl` (bulky fields trimmed). `fold(events)` derives the agent-side record fields from `PostToolUse` events only:

Bob tools	Record field
<code>read_file</code> <code>glob</code> <code>grep</code> <code>list_files</code> <code>GetSymbolsOverview</code> <code>FindSymbol</code> <code>FindReferencingSymbols</code>	<code>read[]</code> <code>kind=file</code> (path or pattern)
<code>write_file</code> <code>apply_diff</code> <code>insert_content</code> <code>search_and_replace</code>	<code>wrote[]</code>
<code>execute_command</code>	<code>commands[]</code> ; <code>`npm</code>
<code>use_mcp_tool</code> <code>access_mcp_resource</code>	<code>read[]</code> <code>kind=mcp</code> (server/tool)
<code>fetch_url</code> <code>web_fetch</code> <code>browser_action</code>	<code>read[]</code> <code>kind=url</code>

3.3 Session manager (`session.py`, `cli.py`)

Hooks installed by `origit init` (`.bob/settings.json`) call the dispatcher `.bob/hooks/origit-hook.sh <subcommand>`, which locates the CLI (`$ORIGIT_BIN`, `git config origit.bin`, `PATH`, `repo-local venv`) and never fails.

Hook	Command	What happens
SessionStart	<code>origit session start</code>	writes <code>.origit/session.json</code> <code>{id, number, started_at, run: 0, head, dirty{path: sha256}, runs[]}</code> ; <code>number</code> = next free display id (see 3.5)
UserPromptSubmit	<code>origit run start</code>	if the working tree is dirty and <code>origit.autocommit</code> is on, commits those edits first as <code>human: edits before session #n run m</code> with <code>ORIGIT_ACTOR=human</code> (so human and agent work never share a record); then <code>run += 1</code> , stores the prompt, appends the event
PostToolUse (matcher <code>.*</code>)	<code>origit trace</code>	appends the event
Stop	<code>origit run end</code>	stages everything except <code>.origit/</code> state, commits with subject <code>bob: <first line of the prompt, ≤64 chars> [session #n run m]</code> and <code>body</code> = Bob's own summary; the git hooks fold and attach the record (3.4). With nothing to commit, the folded record is stored under <code>.origit/runs/<session>-<run>.json</code> and the trace is cleared. Never blocks Bob (always exits 0).

`origit record fold` (pre-commit) runs the tests when `ORIGIT_RUN_TESTS=1` (set by `run end`): `git config origit.testCommand`, else `npm test` if `package.json` has a test script, else `pytest`; failed tests are recorded, never gated. `git config origit.mode` (or `ORIGIT_MODE`) fills `actor.mode`; `origit.approver` fills `approver`.

Human commits made by hand still get a record (`actor: human`) through the same git hooks. Agent commits without a trace can be refused with `ORIGIT_REQUIRE_TRACE=1`.

3.4 Storage (notes.py, git hooks)

.githooks/pre-commit → origit record fold → .origit/pending-record.json; .githooks/post-commit → origit record attach → git notes --ref=origit add -f on HEAD, then the trace is cleared. origit init sets core.hooksPath=.githooks, origit.bin, origit.autocommit=true, notes.rewriteRef=refs/notes/origit (amend/rebase carry notes). Notes travel with git push <remote> main refs/notes/origit. History is never rewritten by Origit; older commits simply have no record.

3.5 Display ids and grouping (sessions.py)

Session ids are 32-hex hashes; people need #42. number_sessions(commits, base) uses session.number from the record when present, otherwise assigns numbers by first appearance (oldest commit first) starting at base, skipping taken numbers. base comes from the committed file .origit/config.json {"session_base": 42} (origit init --session-base 42), read from the working tree by the CLI and at HEAD by the console. group(commits, base) yields sessions newest first with their runs; human commits and commits without records form pseudo-sessions.

3.6 Taint (taint.py)

query(commits, needle) with commits newest first. Needle: package name (fast-pay-utils, any version) or spec (fast-pay-utils@2.1.0), a file path, or a 64-hex sha256. Direct match: added_deps by name(+version), read[] kind=pkg, any read[].ref equal to the needle or (bare package name) containing it as a path segment (node_modules/<name>/README.md), read[].sha256, wrote[]. Propagation, oldest → newest: a commit that read or wrote a file which an already affected commit wrote is affected too (propagated:<path>). Output: affected[] (with matched[] reasons), sessions, files_written, approvers, first_read {session, at, commit}, rollback_commit (nearest clean commit older than the oldest affected), clean[]. Pure function, O(commits); milliseconds on the demo repo. The CLI adds session_labels.

3.7 Pre-filter (prefilter.py)

Deterministic, zero Bobcoins, runs on every commit/push. Rules → findings {asi, title, severity, cwe, evidence, ref}:

Rule	ASI	Severity	CWE
new dependency	ASI04	low	CWE-829
dependency added without reading its docs	ASI04	medium	CWE-829
external read (url/mcp)	ASI01	informational	CWE-829
commands executed	ASI05	informational	CWE-94
agent config changed (.bob/ , AGENTS.md , CLAUDE.md , rules, skills)	ASI01	medium	CWE-829
invisible text in anything read: Unicode tags U+E0000–E007F (decoded and quoted), zero-width, bidi overrides	ASI01	high	CWE-506

Rule	ASI	Severity	CWE
secret/credential file touched (<code>.env*</code> , <code>*.pem</code> , <code>id_rsa</code> , <code>*.key</code> , <code>secrets.*</code>)	ASI03	medium	CWE-200

`decode_unicode_tags()` maps `U+E0020–E007E` back to ASCII; this is how the hidden instruction in the demo README is revealed.

3.8 CLI reference

Command	Purpose
<code>origit init [--force]</code> <code>[--session-base N]</code>	install <code>.bob/</code> (hooks, modes, <code>mcp.json</code> with the absolute CLI path, skill), <code>.githooks/</code> , git config, <code>.gitignore</code> entries
<code>origit session start</code> <code>/ origit session</code> <code>status [--json]</code>	hook consumer / current session for the status bar <code>{id, number, label, run, recording}</code>
<code>origit run start /</code> <code>origit run end</code>	hook consumers (3.3); <code>session-commit</code> is a hidden alias of <code>run end</code>
<code>origit trace</code>	hook consumer (append event)
<code>origit record</code> <code>fold attach</code>	called by the git hooks
<code>origit log [--json]</code> <code>[--flat] [--range]</code>	sessions → runs; JSON: <code>{repo, head, base, sessions:[{id, number, label, actor, mode, started_at, ended_at, runs:[{run, sha, subject, n_read, n_wrote, n_deps, tests, approver, prefilter:{needs_review, max_severity, reasons}, record}]}}</code>
<code>origit show <commit></code> <code>[--json]</code>	the record; JSON <code>{sha, subject, verified, record}</code>
<code>origit taint <needle></code> <code>[--json] [--range]</code>	3.6
<code>origit prefilter</code> <code>[<commit>]</code>	3.7 on a commit, reading the files still present in the tree
<code>origit export [--range]</code>	evidence pack <code>{repo, range, exported_at, head, commits:[{sha, subject, author, date, parents, files, record}]}</code>
<code>origit mcp</code>	MCP server on stdio (4.3)

4. Bob IDE integration

`origit init` writes the following into the repository's `.bob/` (templates in `origit/origit/templates/bob/`):

4.1 Hooks — `settings.json`

Four hooks (3.3). Timeouts: 10 s session start, 30 s run start, 5 s trace, 120 s run end (the tests run inside it). Bob Shell honours the same file, so headless runs are traced identically.

4.2 Modes — `custom_modes.yaml`

- **origit-build**: build features with provenance; edit restricted to `^src/.*` via `fileRegex`; rules (`rules-origit-build/`) ask Bob to read library docs with `read_file` before using them, pin dependency versions, end with an "Origit declaration", and not to run `git commit` (Origit commits the run).
- **origit-review**: read-only reviewer; rules = the condensed OWASP Top 10 for Agentic Applications rulebook (`rules-origit-review/01-...md`) and evidence rules; prompt "Review commit HEAD with Origit" → `origit show HEAD --json`, `git show`, reads every input, answers with one JSON object with ten categories `{asi, status: finding|checked-clean|not-applicable, severity, cwe, evidence, rationale}`. Groups: `read`, `execute`, `todo` (no edit).

4.3 MCP server — `mcp.json` → **origit mcp** (`mcp.py`, written by Bob against `tests/test_mcp.py`)

Stdio, JSON-RPC 2.0, newline-delimited; methods `initialize` (protocol 2024-11-05), `notifications/initialized`, `ping`, `tools/list`, `tools/call`; unknown → -32601. Tools: `origit_taint {needle}`, `origit_show {commit=HEAD}`, `origit_log {limit=20}`; all answered by the core modules (the only subprocess is `git rev-parse`). `alwaysAllow` lists the three tools. Verified with Bob Shell: the question "which commits read fast-pay-utils and what did they write?" was answered in three tool calls.

4.4 Skill — `skills/origit/SKILL.md`

Activated automatically for provenance questions; prefers the MCP tools, falls back to the CLI; always adds "affected means matched by provenance, not confirmed compromise"; never modifies records.

4.5 Headless runs — `tools/bob/run-task.sh <workspace> <mode> <slug> "<prompt>"`

Runs `bob run --format json` with the hooks live, writes `bob_sessions/<slug>.json` (task id, cost, duration, tool calls, last message). Parallel tasks run in separate git worktrees so traces never mix (see `AGENTS.md`).

5. Demo repository

`origit-demo-payments-api` (submodule `demo/payments-api` in the product repo; snapshots in `demo/evidence/`). A small Express/Jest payments API (synthetic data, test PAN 4111111111111111). `packages/fast-pay-utils/2.1.0` is the compromised release: `initializeTelemetry()` runs at import time and posts `process.env` and `.env` to `localhost:8080` (never leaves the machine), hidden by ~300 spaces of indentation; `README.md` line 2 and the JSDoc carry a Unicode-tag instruction telling agents to call it around `processPayment()`. `ADVISORY.md` is a GHSA-style synthetic advisory (CVSS 3.1 8.2 HIGH).

History (18 commits, 17 with records): scaffold (#42), payout export via `fast-pay-utils@2.1.0` (#42, first read), health endpoint, payment utils (#43), scheduled export (#44), advisory (#45), two Saturday runs (#46, #47), the hooks upgrade (human), settlement retry logic (#48, clean), PAN

masking in export logs (#49, propagated). `origit taint fast-pay-utils` → 7 affected commits, sessions #42-#47 and #49, roll back to 17c436c; #48 stays clean.

6. Origit Console

FastAPI + Jinja2, Python 3.12, git, Node 24 + Bob Shell in the image. ~1,700 lines in `app/`.

6.1 Storage and ingest (`gitrepo.py`, `pushes.py`)

- Bare repositories under `ORIGIT_DATA_DIR/repos/<org>/<name>.git` (on the server: host path `/srv/origit/repos`, so `git push` over SSH lands directly in the console's data). A `post-receive` hook installed by the console POSTs `{org, repo, updates[{old,new,ref}]}` to `/api/hooks/post-receive` with the console token.
- Mirrors: `POST /api/repos {"org","name","url"}` clones a public repository; `POST .../sync` fetches branches and `refs/notes/origit`.
- Per-repo metadata `origit-console.json` in the bare repo (`description`, `kind`, `source`, `plan`, `created_at`). State under `ORIGIT_DATA_DIR/state/<org>/<name>/`: `pushes.jsonl`, `evidence/<sha>.json`, `push-evidence/<push id>.json`, `drafts/<needle>.json`.
- Reads that are not in the tree (e.g. `node_modules/<pkg>/README.md`) are resolved by content hash: the record carries the sha256 of what the agent saw and the console finds a committed blob with the same hash (the vendored package README).
- Records are read through the vendored core (`origit.notes`), hashes re-verified on every view.

6.2 Plans

`plan` per repository: `free` (commits with their record, file provenance, taint, evidence pack) or `enterprise` (label "Business": pre-filter, Bob Review, Security tab, Article 14 draft).

6.3 Bob Review (`bobshell.py`, `analysis.py`, `prompts/`)

1. **Per run** (`review_commit`): a throwaway workspace gets `record.json`, `prefilter.json`, the diff (`trim_diff` drops lockfiles, vendored, compiled and binary files, caps at 25 kB), the text of every read input (`reads/*.txt`, hidden Unicode-tag text decoded at the top) and the rulebook; `bob run --format json --mode ask` with `prompts/asi-reviewer.md`; the answer is parsed (`_extract_json`: fenced block, balanced braces, shell-escape repair), normalised to exactly ten categories, cached as `evidence/<sha>.json {summary, categories[], stats, elapsed_s, raw, parsed_ok}`.
2. **Per push** (`review_push`): once every run in a push has evidence, `push.json` (records summarised, findings with short quotes) goes to Bob with `prompts/push-reviewer.md`; result `{summary, first_action, verdicts[10]}` cached as `push-evidence/<push id>.json`.
3. **Aggregation** (`analysis.push_evidence`): per push, per ASI: citations from Bob (or from the pre-filter while Bob is pending), status finding | checked-clean | not-applicable | pending, max severity, sessions. `CHECKS` gives each ASI a plain-language name and question.
4. **Triggering**: post-receive and sync schedule `run_reviews()` in the background for every new commit with a record (plan `enterprise`), sequential, one `bob run` at a time, each capped by `BOB_MAX_COST`; push summaries follow. Backfill: `POST .../review-all`, `POST .../review-pushes[?push_id=...&force=true]`; status: `GET .../review-status`, `GET .../push-evidence`.

5. **Seeding:** `scripts/seed-from-state.sh <state-dir>` copies evidence, push evidence and drafts into `seed/`, which ships in the image; `load_*` fall back to `seed/` so a fresh deploy shows Bob's output immediately (source: `seed`). `scripts/reparse-evidence.py` re-parses cached raw answers without calling Bob.
6. **Article 14** (`draft_art14`): on a taint query, `taint.json` (+ `ADVISORY.md` if the repo has one) go to Bob with `prompts/art14-early-warning.md`; the draft is cached per needle. A deterministic template (`/draft-art14/template`) exists for when Bob is unavailable.

Display policy on the Security page (deterministic, the evidence itself is never edited): a *finding* is medium or worse on an agent run; everything else is a *note*, still visible on the commit page.

6.4 Pages

`/` landing (numbers computed live from the featured repo) · `/explore` · `/<org>` · `/<org>/<name>` code browser (README rendered, invisible characters marked) · `.../blob/<path>` (readers and writers of the file, hidden text decoded) · `.../commits` (session labels, pre-filter reason tags) · `.../commit/<sha>` (record, hash verification, pre-filter, Bob evidence, diff) · `.../pushes` (one card per push: Bob summary, first action, findings, checked clean / not relevant, commits) · `.../taint?q=` (affected red / clean green, files split into primary and "also touched", Article 14 draft) · `.../security` (four numbers, what needs attention, ten check tiles, one line per push) · `.../record/<sha>.json`.

6.5 API

Read (public): `GET /api/health`, `/api/repos`, `/api/{org}/{name}`, `.../commits`, `.../commit/{sha}`, `.../taint?q=`, `.../security`, `.../pushes`, `.../push-evidence`, `.../review-status`, `.../export` (evidence pack download), `.../draft-art14/template?q=`. Interactive docs at `/api/docs`. Mutating (header `X-Origit-Token`): `POST /api/repos`, `POST .../plan`, `POST .../sync`, `POST /api/hooks/post-receive`, `POST .../review-all`, `POST .../review-pushes`, `POST .../review/{sha}[?force]`, `POST .../draft-art14?q=[&force]`. With `CONSOLE_READONLY=1` the first three return 403 (read-only demo); Bob actions stay token-gated.

6.6 Configuration (environment, `.env` never committed)

`ORIGIT_DATA_DIR` (`./data`) · `CONSOLE_TOKEN` · `BOB_API_KEY` (Bob Shell key with Inference scope; on the demo server a teammate's key) · `BOB_MAX_COST` (0.5) · `BOB_MAX_TURNS` (6; push reviews use ≥ 10) · `BOB_TIMEOUT` (240 s) · `PUBLIC_URL` · `GIT_SSH_HOST` (display only) · `GIT_SSH_ROOT` · `HOOK_URL` · `GITHUB_URL` · `CONSOLE_READONLY` (0) · `DEMO_ACCOUNT` (shows " · Business plan · demo" in the top bar).

6.7 Deployment

`deploy/Dockerfile` (python:3.12-slim + git + Node 24 + Bob Shell installer, best effort), `deploy/docker-compose.yml` (project `origit`, container `origit-console`, joins the shared Caddy network, mounts `/srv/origit/repos` and a state volume), `deploy/origit-uk.caddy` (reverse proxy drop-in), `deploy/publish.sh` (rsync sources, scp `.env`, docker compose build/up, Caddy reload, health check). Local: `uv venv .venv && uv pip install -e ".[dev]" && bash scripts/dev.sh`. Tests: 23 backend tests (`backend_tests/`), run against a temporary import of the demo repo.

7. VS Code extension "Origit for Bob IDE"

`extensions/origit-vscode/` — TypeScript, no bundler, no runtime dependencies; scaffolded by Bob (task 10), reviewed and fixed by hand; packaged as `origit-vscode-0.1.0.vsix` (`npx @vscode/vsce package --allow-missing-repository`). Install: Extensions → ... → Install from VSIX, or `bobide --install-extension origit-vscode-0.1.0.vsix`. Bob IDE is a VS Code fork (engine 1.126, extensions from `open-vsx`), so the standard extension API applies; nothing in Bob IDE itself is modified.

- **View** `origit.sessions` ("ORIGIT") under the Source Control container: sessions (#42 · Bob IDE · `origit-build` · 3 runs, robot/person icon) → runs/commits (<sha> <subject>, description n read · n wrote · n deps, severity dot from the pre-filter: red high/critical, orange medium, yellow low, blue informational, green none) → groups Read / Wrote / Dependencies / Commands, plus Approver, Tests, Record sha256, Prompt.
 - **Commands:** `Origit: Taint...` (input box → affected commits get a red error icon; message with count, session labels and roll-back commit), `Origit: Clear taint`, `Origit: Refresh` (view title), `Origit: Show record` (context menu → JSON document).
 - **Status bar:** `$(record) Origit: session #48 run 1 · recording from origit session status -json, fallback .origit/session.json, else Origit: idle.`
 - **Data source only:** `origit log --json, origit taint <needle> --json, origit show <sha> --json, origit session status --json`, run with `child_process.execFile` in the first workspace folder. CLI resolution: setting `origit.binary` → `git config origit.bin` → `origit` on PATH. Refresh every 15 s and on changes under `.origit/**` and `.git/refs/notes/**`. Errors show as a single tree item.
-

8. Security model and claims

What the record proves: the hash binds the record to its content at commit time; tampering with a note is detectable (`verify`); a record was captured by hooks at the time, not reconstructed. What it does not prove: that the code is safe, that nothing was missed, that the agent could not evade tracing (an agent with shell access can act outside the traced tools; commands are recorded, their side effects are inferred from the commit). Bob's output is evidence for a human, never a gate decision; where it is noisy (it also flags here-doc writes and human hook edits) it is shown as is.

Claims discipline (`docs/positioning.md`): say "agent provenance layer for git", "records what the agent read", "answers the CRA 24-hour question in seconds", "cited evidence per ASI category", "open-core", "Bob-native first, agent-agnostic by design"; never "proves the code is safe", "nothing was missed", "replaces git", "works with any agent" unqualified, "PCI-DSS compliant", "Bob guarantees".

Secrets: `.env`, `bob-*.json`, `apikey*` are git-ignored in every repository; keys live only in `.env` on the server (copied by `publish.sh`). Data: synthetic only; the exfiltration in the demo package targets `localhost` and is never executed by the demo app's tests.

9. Operations

- **Add Orit to a repository:** `pip install -e orit/, cd repo && orit init --session-base 1`, work in Bob IDE; each run commits itself; push with `git push origin main` refs/notes/orit.
- **Answer an advisory:** `orit taint <package>` (CLI, offline) or the console taint page; "Draft with Bob" for the Article 14 early warning; `orit export > evidence.json` for the evidence pack.
- **History rewrites:** never rewrite recorded history; if unavoidable, `git notes --ref=orit copy <old> <new>` per commit and push the notes ref again.
- **Console upgrade:** bump `vendor/orit`, run the backend tests, `bash deploy/publish.sh`; new Bob output on the server can be pulled into `seed/` with `docker cp orit-console:/data/state ./state-copy && bash scripts/seed-from-state.sh ./state-copy`.
- **Costs:** a per-run review costs 0.03–0.11 Bobcoins, a push summary 0.02–0.05, an Article 14 draft ≈0.05 (Bob Shell, September 2026 pricing).

Related documents: `README.md` (pitch and quick start), `AGENTS.md` (conventions for agents working on Orit), `docs/asi-mapping.md`, `docs/cra-article-14.md`, `docs/demo-script.md`, `docs/roadmap.md`, `docs/STATUS.md`.